Simple Linear Regression

What I'm Doing

I implemented a simple linear regression with scikit-learn, using engine size to predict **CO_2 emissions** from Canadian vehicle fuel data. I split the dataset 80/20 for training and testing, then evaluated the model with MAE, MSE, and R^2.

---

1. Setting Up

I start by importing the essential libraries for data manipulation, numpy, and visualization.

```
%pip install matplotlib pandas numpy scikit-learn
import matplotlib.pyplot as plt
import pandas as pd
import pylab as pl
import numpy as np
%matplotlib inline
```

download dataset from IBM Object Storage using `!curl -o` (Linux: `!wget -O`)

```
!curl -o FuelConsumption.csv https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-ML0101EN-SkillsNetwork/labs/Module%202/data/FuelConsumptionCo2.csv
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed

  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
  0     0    0     0    0     0      0      0 --:--:--  0:00:01 --:--:--     0
100 72629  100 72629    0     0  31942      0  0:00:02  0:00:02 --:--:-- 31981
```

---

2. Getting to Know the Dataset

Then I load and inspect the first few rows.

```
df = pd.read_csv("FuelConsumption.csv")
df.head()
```

	MODELYEAR	MAKE	MODEL	VEHICLECLASS	ENGINESIZE	CYLINDERS	TRANSMISSION	FUELT
0	2014	ACURA	ILX	COMPACT	2.0	4	AS5	
1	2014	ACURA	ILX	COMPACT	2.4	4	M6	
2	2014	ACURA	ILX HYBRID	COMPACT	1.5	4	AV7	
3	2014	ACURA	MDX 4WD	SUV - SMALL	3.5	6	AS6	
4	2014	ACURA	RDX AWD	SUV - SMALL	3.5	6	AS6	

`Result:` The dataset contains model year, make, model, vehicle class, engine specs, fuel consumption ratings, and the target - CO_2 emissions in g/km. Dataset source

A quick statistical summary of each numerical column.

```
df.describe()
```

	MODELYEAR	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_CITY	FUELCONSUMPTION_HWY
count	1067.0	1067.000000	1067.000000	1067.000000	1067.000000
mean	2014.0	3.346298	5.794752	13.296532	9.474602
std	0.0	1.415895	1.797447	4.101253	2.794510
min	2014.0	1.000000	3.000000	4.600000	4.900000
25%	2014.0	2.000000	4.000000	10.250000	7.500000
50%	2014.0	3.400000	6.000000	12.600000	8.800000
75%	2014.0	4.300000	8.000000	15.550000	10.850000
max	2014.0	8.400000	12.000000	30.200000	20.500000

---

3. Exploring the Data

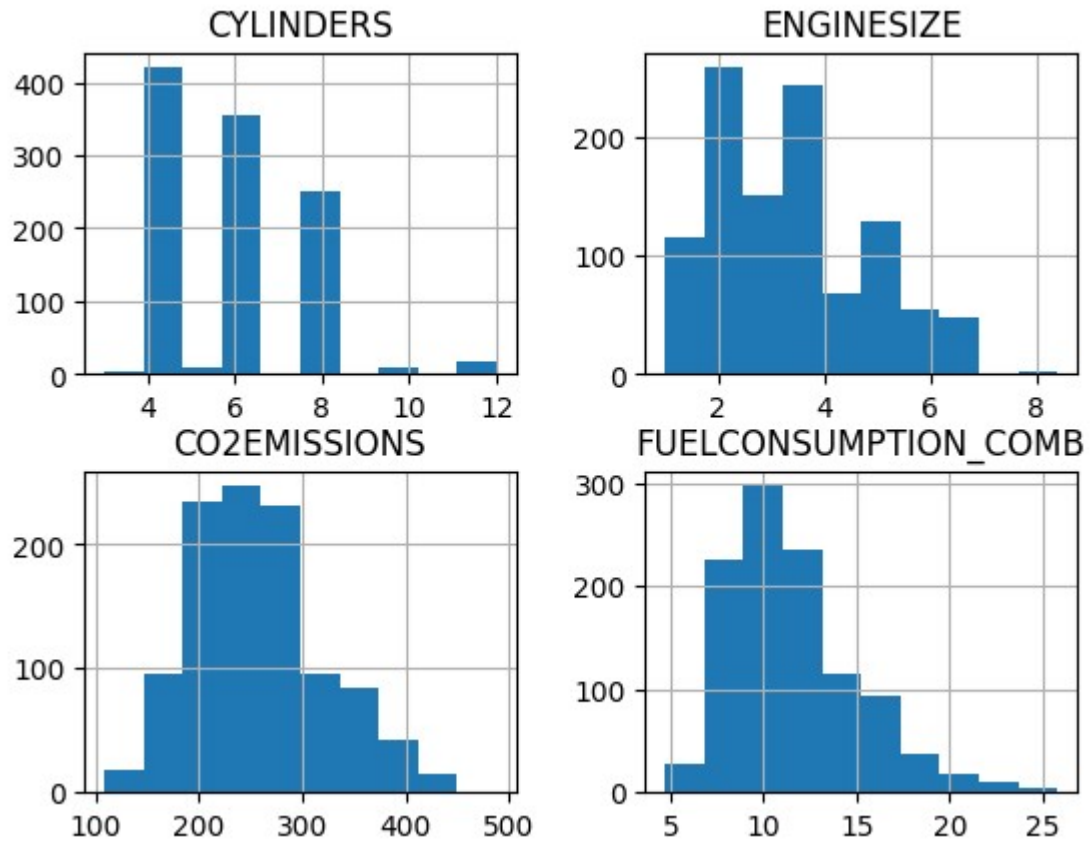
I narrow my focus to the four most relevant features for this regression task.

```
cdf = df[['ENGINESIZE','CYLINDERS','FUELCONSUMPTION_COMB','CO2EMISSIONS']]
cdf.head(9)
```

	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267

Histograms give a quick sense of how each feature is distributed.

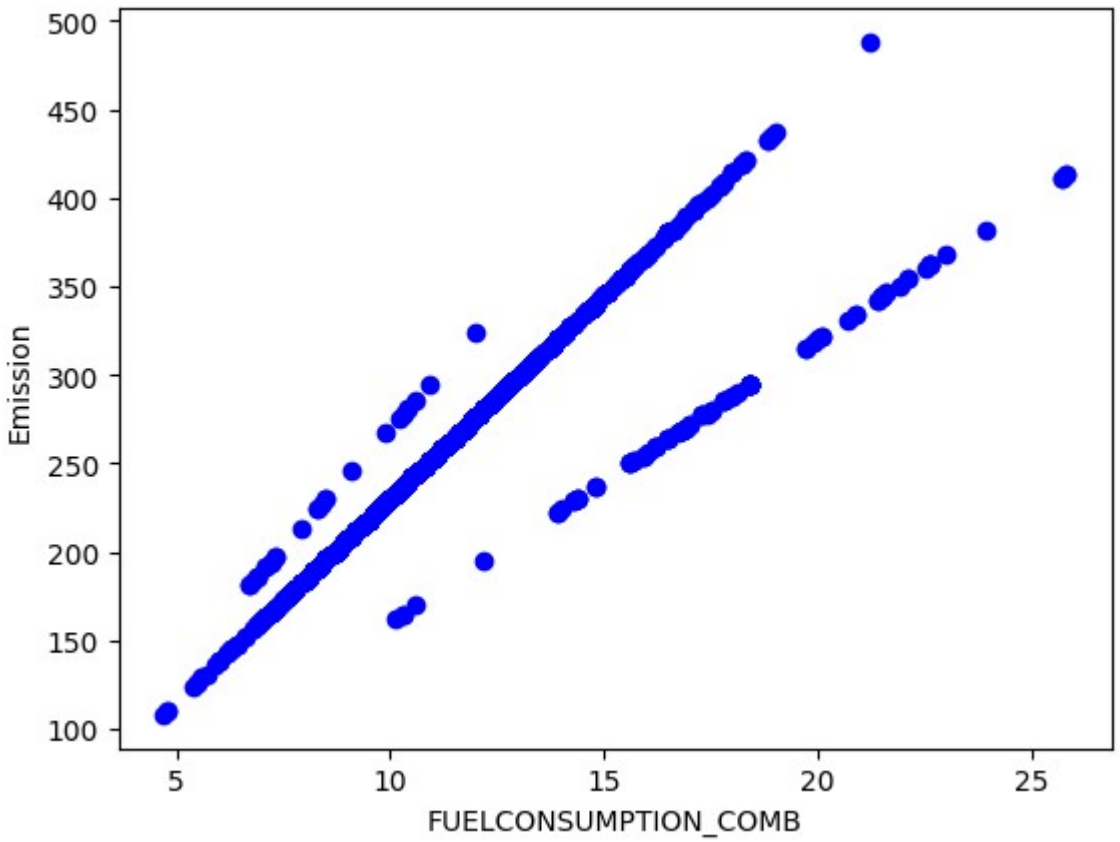
```
viz = cdf[['CYLINDERS','ENGINESIZE','CO2EMISSIONS','FUELCONSUMPTION_COMB']]
viz.hist()
plt.show()
```



`Result:` Four histograms display. Engine size clusters around 2.0–4.0 L, cylinders peak at 4, 6, and 8 combined fuel consumption is right-skewed, and CO_2 emissions form a roughly bell-shaped curve centered near 250 g/km.

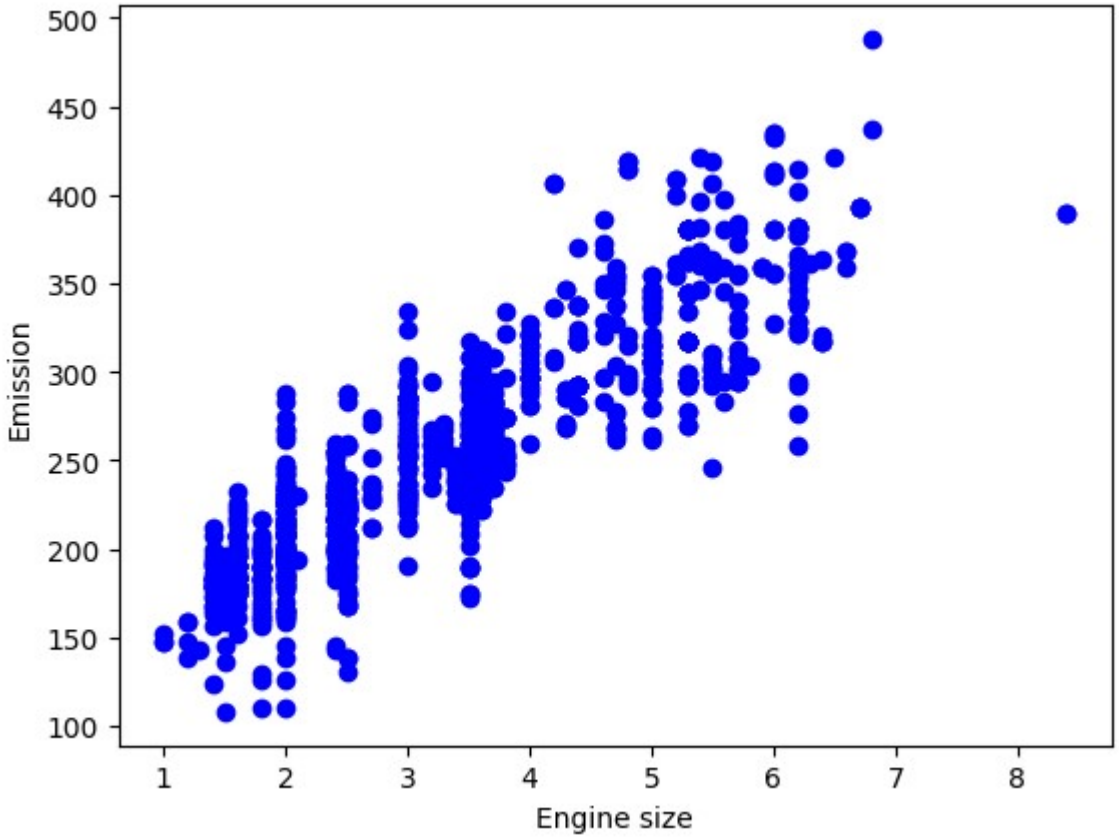
Now I check the relationship between each feature and the target variable using scatter plots **to predict emissions**.

```
plt.scatter(cdf.FUELCONSUMPTION_COMB, cdf.CO2EMISSIONS, color='blue')
plt.xlabel("FUELCONSUMPTION_COMB")
plt.ylabel("Emission")
plt.show()
```



`Result:` A strong **positive linear relationship** - as fuel consumption increases, CO_2 emissions rise proportionally.

```
plt.scatter(cdf.ENGINESIZE, cdf.CO2EMISSIONS, color='blue')
plt.xlabel("Engine size")
plt.ylabel("Emission")
plt.show()
```

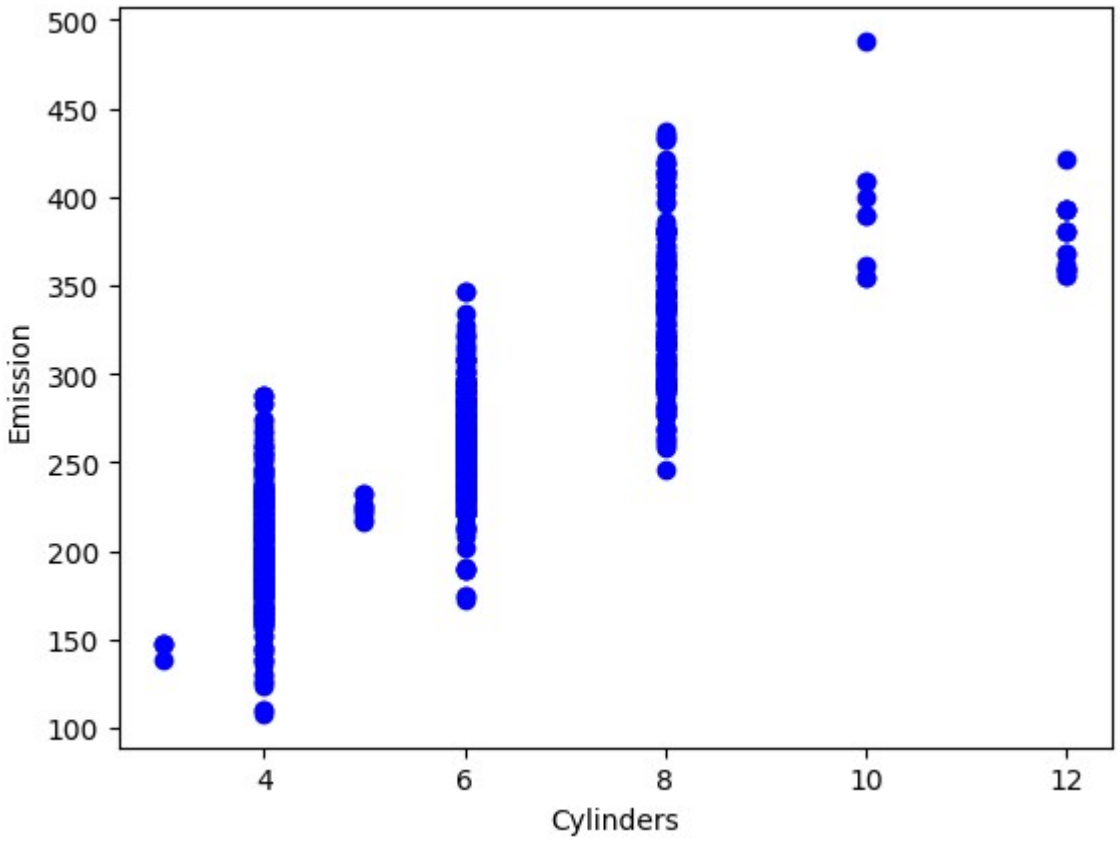


`Result:` Another positive trend, though with more scatter. **Larger engines generally produce more CO_2**, but the **relationship is noisier** than fuel consumption.

Exercise: Cylinders vs. Emission

Let's visualize how cylinder count correlates with CO_2 emissions.

```
plt.scatter(cdf.CYLINDERS, cdf.CO2EMISSIONS, color='blue')
plt.xlabel("Cylinders")
plt.ylabel("Emission")
plt.show()
```



Result: A clear upward trend - vehicles with more cylinders emit more CO_2. The data clusters at specific cylinder counts (4, 6, 8, etc.), making this a reasonable but coarser predictor than engine size or fuel consumption.

---

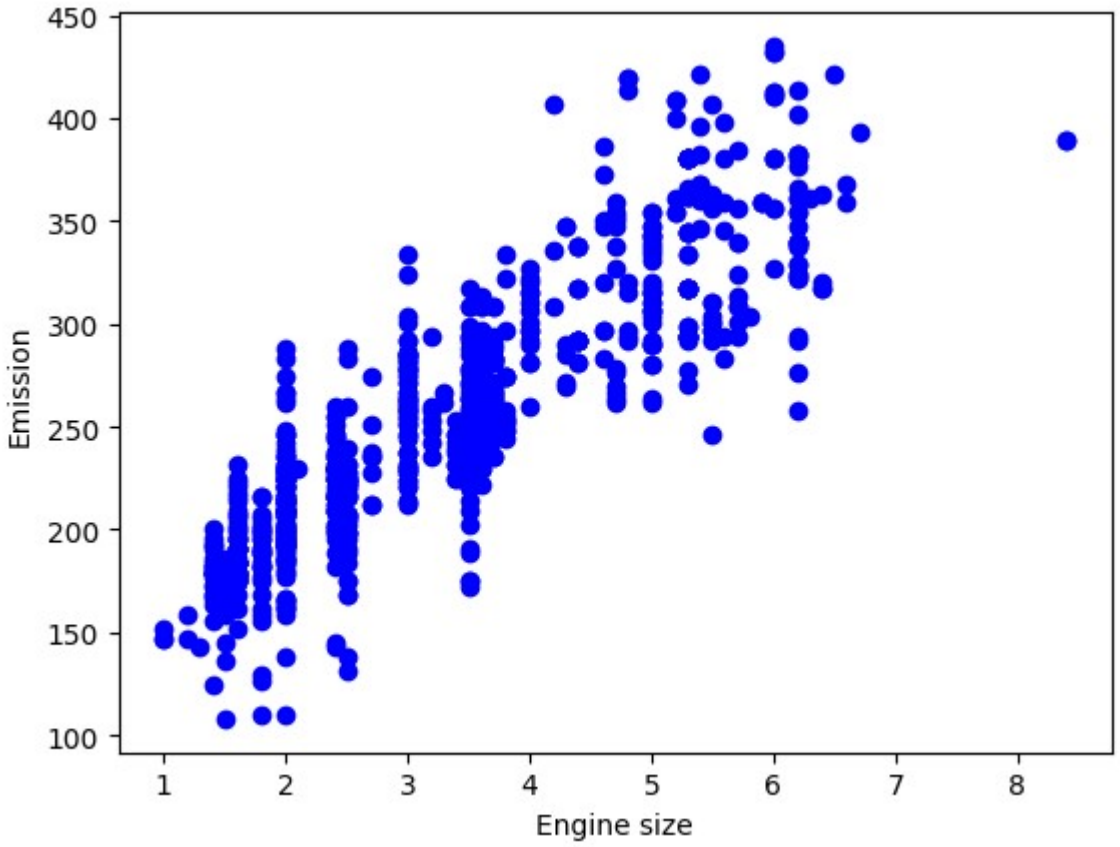
4. Train/Test Split

I split the data so the model trains on 80% of observations and tests on the remaining 20%. This ensures I evaluate performance on data the model has never seen by creating a mask to select random rows using **np.random.rand()** function:

```
msk = np.random.rand(len(df)) < 0.8
train = cdf[msk]
test = cdf[~msk]
```

Let's visualize the training subset.

```
plt.scatter(train.ENGINESIZE, train.CO2EMISSIONS, color='blue')
plt.xlabel("Engine size")
plt.ylabel("Emission")
plt.show()
```



Result: The training data preserves the same positive trend observed in the full dataset.

---

5. Building the Simple Linear Regression Model

I use scikit-learn's `LinearRegression` to fit a line through the training data, minimizing the residual sum of squares.

```
from sklearn import linear_model

regr = linear_model.LinearRegression()
train_x = np.asanyarray(train[['ENGINESIZE']])
train_y = np.asanyarray(train[['CO2EMISSIONS']])
regr.fit(train_x, train_y)

print('Coefficients: ', regr.coef_)
print('Intercept: ', regr.intercept_)
```

```
Coefficients:  [[38.41986307]]
Intercept:  [127.24947105]
```

The fitted line is:

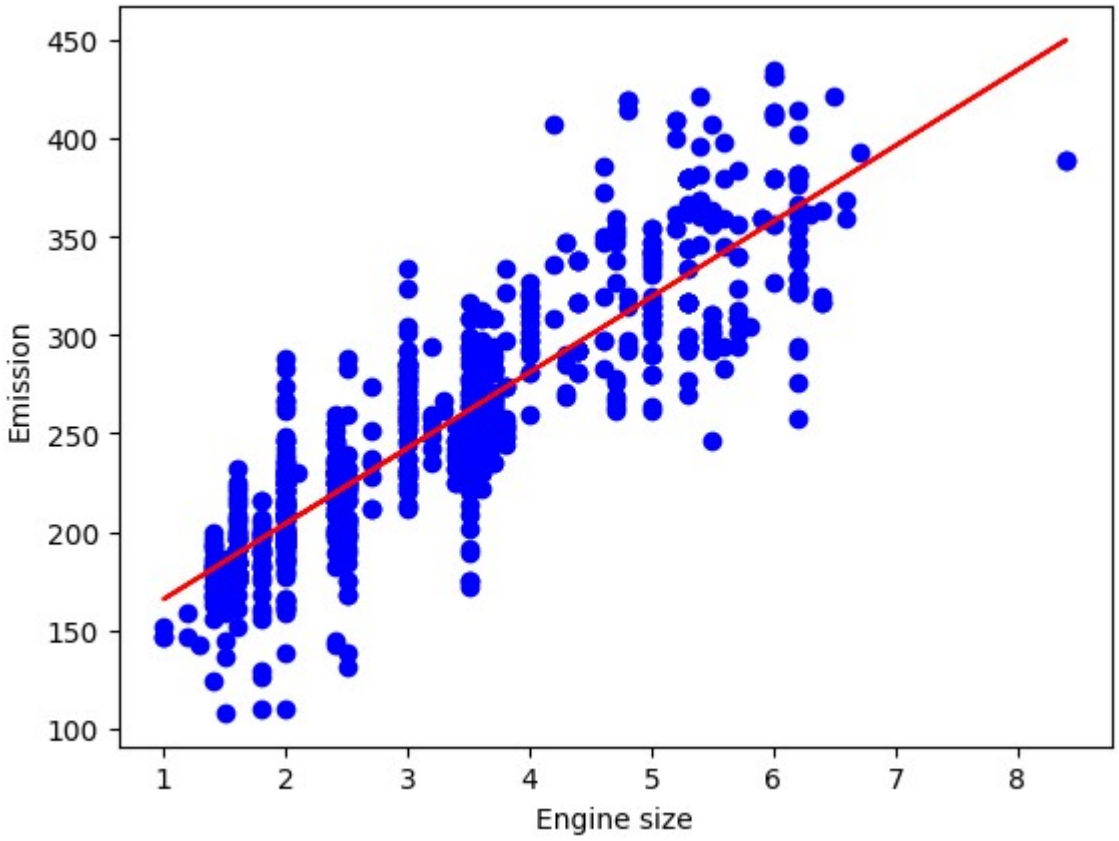
$$\text{CO2 Emissions} = 38.42 \times \text{Engine Size} + 127.25$$

Each additional litre of engine size adds roughly **39 g/km** of CO_2, on average.

Plotting the regression line over the training data shows how well it captures the trend.

```
plt.scatter(train.ENGINESIZE, train.CO2EMISSIONS, color='blue')
```

```
plt.plot(train_x, regr.coef_[0][0]*train_x + regr.intercept_[0], '-r')
plt.xlabel("Engine size")
plt.ylabel("Emission")
plt.show()
```



Result: The red regression line cuts through the scatter, **rising steadily with engine size**. While some variance remains (points spread around the line), the overall linear trend is clear.

---

6. Evaluating the Model

I test the model on the held-out 20% and compute three standard metrics.

```
from sklearn.metrics import r2_score

test_x = np.asanyarray(test[['ENGINESIZE']])
test_y = np.asanyarray(test[['CO2EMISSIONS']])
test_y_ = regr.predict(test_x)

print("Mean absolute error: %.2f" % np.mean(np.absolute(test_y_ - test_y)))
print("Residual sum of squares (MSE): %.2f" % np.mean((test_y_ - test_y) ** 2))
print("R2-score: %.2f" % r2_score(test_y, test_y_))
```

```
Mean absolute error: 24.80
Residual sum of squares (MSE): 1034.49
R2-score: 0.77
```

- **MAE (24.80 g/km):** On average, the model's predictions are off by about 25 g/km.
- **MSE (1034.49):** Squared errors penalize larger deviations.

- **R^2 (0.77): Engine size** explains 77% of the variance in CO_2 emissions - a solid fit for a single feature. (best fit is **1.0**)

---

7. Exercise: Using **Fuel Consumption** as a Feature

Now I repeat the entire workflow using **FUELCONSUMPTION_COMB** as the predictor instead of engine size. This lets me compare which single feature performs better.

Step 1 - Prepare the training and testing data:

```
train_x = train[["FUELCONSUMPTION_COMB"]]
test_x = test[["FUELCONSUMPTION_COMB"]]
```

Lets see what the evaluation metrics are if we trained a regression model using the `FUELCONSUMPTION_COMB` feature.

Start by selecting `FUELCONSUMPTION_COMB` as the train_x data from the `train` dataframe, then select `FUELCONSUMPTION_COMB` as the test_x data from the `test` dataframe

Step 2 - Train the model:

```
regr = linear_model.LinearRegression()
regr.fit(train_x, train_y)
```

▾ LinearRegression

▾ Parameters

	fit_intercept	True
	copy_X	True
	tol	1e-06
	n_jobs	None
	positive	False

▾ Fitted attributes

Name	Type	Value
coef_	ndarray[float64](1, 1)	[[15.87]]
feature_names_in_	ndarray[object](1,)	['FUELCONSUMPTION_COMB']
intercept_	ndarray[float64](1,)	[71.82]
n_features_in_	int	1
rank_	int	1
singular_	ndarray[float64](1,)	[102.33]

Step 3 - Make predictions:

```
predictions = regr.predict(test_x)
```

Step 4 - Evaluate:

```
print("Mean Absolute Error: %.2f" % np.mean(np.absolute(predictions - test_y)))
print("Mean Squared Error: %.2f" % np.mean((predictions - test_y) ** 2))
print("R2-score: %.2f" % r2_score(test_y, predictions))
```

```
Mean Absolute Error: 22.15
Mean Squared Error: 873.98
R2-score: 0.80
```

The MAE drops **from ~25 g/km (engine size) to ~22 g/km (fuel consumption)**, and R^2 jumps from 0.77 to 0.80. Combined fuel consumption is a substantially stronger single predictor of CO_2 emissions - which makes intuitive sense, since fuel burned is directly tied to CO_2 produced.

That's the full walkthrough. I used scikit-learn to train and evaluate linear regression models predicting CO_2 emissions, and showed that `FUELCONSUMPTION_COMB` outperformed `ENGINESIZE` - **lowering error** and **raising R^2** (feature selection dramatically affects model performance().